

Available online at www.sciencedirect.com

ScienceDirect

journal homepage: www.elsevier.com/locate/coseComputers
&
Security

Survivable zero trust for cloud computing environments



Luca Ferretti^a, Federico Magnanini^{b,*}, Mauro Andreolini^a,
Michele Colajanni^c

^aDepartment of Physics, Informatics and Mathematics, University of Modena and Reggio Emilia, Italy

^bDepartment of Engineering “Enzo Ferrari”, University of Modena and Reggio Emilia, Italy

^cDepartment of Informatics, Science and Engineering, University of Bologna, Italy

ARTICLE INFO

Article history:

Received 1 February 2021

Revised 8 July 2021

Accepted 28 July 2021

Available online 1 August 2021

Keywords:

Zero trust

Survivability

Distributed systems

Access control

Security

ABSTRACT

The security model relying on the traditional defense of the perimeter cannot protect modern dynamic organizations. The emerging paradigm called zero trust proposes a modern alternative that enforces access control on every request and avoids implicit trust based on the physical location of people and devices. These architectures rely on several trusted components, but existing proposals make the unrealistic assumption that attackers cannot compromise some of them. We overcome these assumptions and present a novel survivable zero trust architecture that can guarantee the necessary security level for cloud computing environments. The proposed architecture guarantees a high level of security and robustness and under specific conditions it can tolerate intrusions and can recover from failures and successful attacks.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

Cyber security based on the defense of the perimeter of an organization is becoming obsolete and unsuitable to organizations that are characterized by “bring your own device” policies, hybrid cloud services, remote working and Internet-of-Things. As a further flaw, the implicit trust granted to internal hosts facilitates lateral movements to attackers who have gained unauthorized access to the inner network and cloud resources (Apuzzese et al., 2017). The emerging security model based on the so-called zero trust (Kindervag, 2010) removes the concept of perimeter by designing architectures that do not grant implicit trust to any host/device because of their physical location. Access to any resource must be previously authenticated and authorized and, to this aim, zero

trust solutions adopt two types of architectural components: the control plane and the data plane (Gilman and Barth, 2017; Rose et al., 2020). The former includes components that collaborate to evaluate access control policies and configure data plane components. The latter elements enforce access control policies to the resources according to the configuration received by the control plane.

As every network flow is considered untrusted, zero trust architectures assume omnipresent internal and external threats (Campbell, 2020; Gilman and Barth, 2017; Kindervag, 2010; Uttecht, 2020). However, existing designs of zero trust architectures in literature assume an untrusted data plane but a trusted control plane (e.g., DeCusatis et al., 2016; Eidle et al., 2017; Lee et al., 2017; Vanickis et al., 2018; Zaheer et al., 2019) that is, they assume that adversaries are confined to the data plane cannot compromise the control plane components.

* Corresponding author.

E-mail addresses: luca.ferretti@unimore.it (L. Ferretti), federico.magnanini@unimore.it (F. Magnanini), mauro.andreolini@unimore.it (M. Andreolini), michele.colajanni@unibo.it (M. Colajanni).

<https://doi.org/10.1016/j.cose.2021.102419>

0167-4048/© 2021 Elsevier Ltd. All rights reserved.

We argue that relying on a trusted control plane is incoherent with zero trust principles as we consider unrealistic to assume that attackers could not compromise control plane components that grant access to system resources. For this reason, we identify possible threats and attacks to zero trust control plane and mitigate them through a novel design that distributes trust within the control plane so that no single component is fully trusted. We evaluate the feasibility of the proposal in cloud computing environments by analyzing performance of the state-of-the-art protocols and components that are required for the implementation of the proposed architecture.

The paper is organized as follows. [Section 2](#) discusses related work. [Section 3](#) introduces the reference zero trust architecture. [Section 4](#) highlights possible attack scenarios. [Section 5](#) overviews the design of the proposed survivable zero trust architecture to prevent the considered attacks. [Section 6](#) describes the adopted technical solutions. [Section 7](#) discusses the security and robustness of the proposed architecture. [Section 8](#) discusses the feasibility of the architecture. [Section 9](#) concludes the paper.

2. Related work

In this work, we propose the first survivable zero trust architecture for cloud computing that satisfies the core principles of zero trust ([Campbell, 2020](#); [Gilman and Barth, 2017](#); [Kindervag, 2010](#); [Uttecht, 2020](#)), namely: the network is assumed to be hostile and characterized by persistent external and insider threats; network flows are not trusted even if they originate from internal networks; each flow must be authenticated and authorized through dynamic access control policies. Existing literature on zero trust assumes that only the data plane can be hostile while the control plane is assumed as trusted (e.g., [DeCusatis et al., 2016](#); [Eidle et al., 2017](#); [Lee et al., 2017](#); [Vanickis et al., 2018](#); [Zaheer et al., 2019](#)). As this assumption contrasts zero trust core principles, we design an architecture that tolerates attacks from external and insider adversaries against the data plane and the control plane.

Some works ([DeCusatis et al., 2016](#); [Eidle et al., 2017](#)) propose zero trust architectures composed by multiple trusted control plane components that do not guarantee security in the case of successful intrusions. These proposals do not store access control policies in a survivable way, hence an adversary which alters them may defeat access control mechanisms. We avoid a similar limitation by storing access control policies in survivable databases that tolerate attacks against policy integrity. As a further limitation, existing proposals assume trusted communication channels among control plane components, hence they cannot guarantee authenticity of control plane configurations. The proposed zero trust architecture secures control plane communications over untrusted channels with well-known public key infrastructure (PKI) and tolerates attacks against PKI components.

Another line of research (e.g., [Lee et al., 2017](#); [Vanickis et al., 2018](#) extend XACML [XACML, 2013](#)) proposes zero trust architectures supporting fuzzy evaluation of dynamic access control policies. These proposals rely on trusted components to evaluate policies and, as a result, they cannot tolerate at-

tacks against the integrity of the policy evaluation process. We present an architecture supporting dynamic access control policies that can even tolerate integrity violations of the policy evaluation process.

The paper in [Zaheer et al. \(2019\)](#) proposes a zero trust architecture that allows perimeterization of microservices, but a similar scenario is quite different than that considered in this work. We refer to users requesting access to resources through managed devices, whereas the other work considers microservices requiring access to other microservices. The difference between the two scenarios is that the entity issuing a request, known as *network agent* ([Gilman and Barth, 2017](#)), in our case is represented by the pair user-device, whereas in [Zaheer et al. \(2019\)](#) refers to microservice identities. This difference is reflected in the different design of the architectural components devoted to network agent authentication. In their proposal, the trusted kernel of the host executing a microservice authenticates requests originating from the microservice. As a result, compromising the host allows the adversary to impersonate any microservice. The architecture proposed in this paper authenticates user and device through intrusion tolerant components that prevent impersonation of a network agent by removing single points of vulnerability.

Recent literature aims to reduce trust assumptions on the control plane by proposing access control architectures based on blockchain ([Dhar and Bose, 2020](#); [Lin et al., 2018](#); [Maesa et al., 2017, 2019](#); [Zhu et al., 2018](#)). For example, the authors in [Dhar and Bose \(2020\)](#) extend the seminal zero trust architecture of [Kindervag \(2010\)](#) by storing and evaluating access control policies on smart contracts. The works of [Maesa et al. \(2017, 2019\)](#); [Zhu et al. \(2018\)](#) extend XACML architecture by designing policy enforcement and decision points as blockchain components. These design choices guarantee the integrity of policies and of the evaluation process even in presence of attacks, but they incur in monetary and computational costs associated with permissionless blockchains. We propose a zero trust architecture with the same security guarantees without incurring in their drawbacks.

The authors in [Lin et al. \(2018\)](#) propose an access control system in Industry 4.0 by adopting a permissioned blockchain that saves monetary costs, but this system considers network agents consisting of devices only and does not include user identities. As a result, it lacks the procedures and components required for user authentication, such as a user database or a single sign-on authentication system. This proposal does not satisfy the zero trust paradigm requiring that both users and devices must be authenticated. Our architecture includes survivable components dedicated to user and device authentication, thus embracing and enhancing zero trust principles completely.

3. Reference zero trust architecture

Multiple architectures have been proposed to instantiate zero trust design principles at different levels of abstraction and coverage of security requirements. We consider a reference architecture that is a subset of the BeyondCorp Zero Trust Architecture (ZTA) ([Ward and Beyer, 2014](#)), which is a full-fledged architecture already deployed in production-ready cloud-based

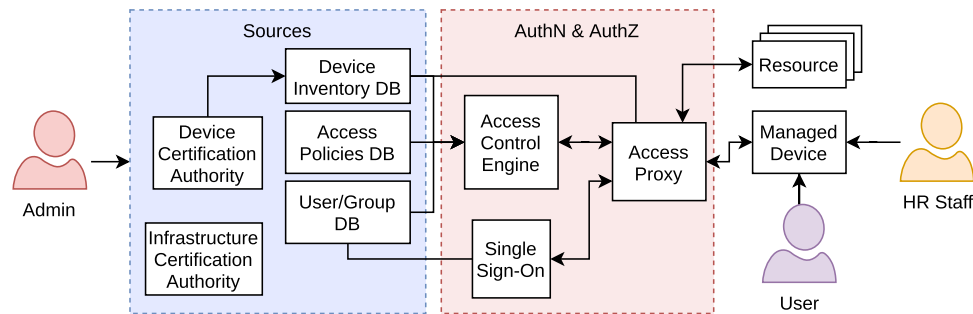


Fig. 1 – Reference zero trust architecture.

environments. In Section 3.1, we show that the architecture has general validity and we discuss how it relates to NIST ZTA (Rose et al., 2020) and XACML architectures (Ardagna et al., 2009; XACML, 2013).

The reference model of the considered ZTA is outlined in Fig. 1. We consider the following components.

- **Administrator:** an employee with the privileged access to the architectural components. He can allow their initial setup and subsequent configurations.
- **User:** a person that requests access to a resource through a managed device.
- **Resource:** applications, services and infrastructures that are subject to access control.
- **Managed device:** device that a user adopts to access a resource; this device is procured and managed by administrators through some device management systems.

A similar architecture requires several components that are responsible for authentication and authorization as reported below.

- **Single Sign-On:** a centralized user authentication service that issues authentication tokens after validating users credentials.
- **Access Control Engine:** a centralized logical component that evaluates access policies and performs authorization by granting or revoking access to a resource.
- **Access Proxy:** a logical component that allows or denies access to resources as instructed by the Access Control Engine.

The Access Control Engine performs authorization by executing a so called *trust algorithm* (Rose et al., 2020) that evaluates access control policies by taking into account several information. It supports the evaluation of the trust level associated to a device-user pair requesting a resource. The sources of information, that are collectively referred as *Sources*, are:

- **Access Policies database:** stores the set of both infrastructure-wide and application-specific policies. A policy is a set of predicates that must be satisfied to access a resource.
- **Device Inventory database:** stores information about devices; the stored information includes device last observed status (e.g., operating system version or installed software)

as well as other additional information (e.g., device owner or device certificates).

- **Device Certification Authority:** issues certificates to managed devices, which are stored in the Device Inventory as well as on devices.
- **Infrastructure Certification Authority:** issues certificates to other ZTA components.
- **User/Group database:** stores information about users and groups. This information is used to evaluate access control policies.

The third role is represented by the **Human Resources Staff** that is granted to write permissions by Administrators over the User/Group database and the Device Inventory.

Administrators grant write permissions to Human Resources staff by defining appropriate access policies in the Access Policies database. Human Resources staff is the only unprivileged actor that is allowed to access the Device Certification Authority, the Device Inventory and the Users/Group databases. A similar grant is denied to normal users.

We highlight that Users, Human Resources Staff and Administrators correspond to the typical roles of an organization. This classification reflects the different levels of trust required to access components and to operate the infrastructure, but it does not limit the general value of the proposal. We can model all personnel that requires access to resources and that must not have access to the User/Group database, the Device Inventory or certification authorities with the User role. Moreover, the considered system model allows personnel to be assigned multiple roles at once (e.g., an administrator that also provisions managed devices).

Let us describe the flow of operations of the reference ZTA. When a new user needs access to resources, the Human Resources staff must complete the *user registration* and *device provisioning* operations. *User registration* is performed by authorized Human Resources staff which adds the new user to the User/Group database. Human Resources staff then completes *device provisioning* by obtaining a new device certificate from the Device Certification Authority, and by inserting a record in the Device Inventory database that contains device information and the device certificate. Once users and their managed devices are registered, they can obtain access to a resource by completing the authentication and authorization flow which involves the following sequence of operations: *device authentication*, *user authentication*, *request authorization* and *resource serv-*

ing. A request for a resource is always received by the Access Proxy which is in charge of *device authentication*. During device authentication the device identifies itself to the Access proxy by presenting the certificate. If the access proxy verifies that the supplied certificate is present in the authentication procedure, then it completes the authentication procedure by executing a public key authentication protocol with the device. If not, the Access proxy terminates the device authentication procedure. Once the device is authenticated, the Access Proxy redirects the user to the single sign-on system for *user authentication*. The user authenticates to the single sign-on system, possibly through multi-factor authentication, and obtains an authentication token attesting the user identity. The single sign-on system redirects the user back to the Access Proxy which forwards the user request, along with the obtained authentication information, to the Access Control Engine for *request authorization*. The Access Control Engine executes the trust algorithm by evaluating the request, the attached authentication information, as well as other data sources, and then determines whether access to the resource is granted or denied. The Access Control Engine finally sends the authorization result to the Access Proxy. If the access is granted, then the Access Proxy proceeds to *resource serving* during which the Access Proxy serves the resource to the user by acting as a reverse proxy. If the access is denied, then the Access Proxy terminates the sequence of operations.

All architectural components communicate over mutually authenticated and confidential channels. Channel authenticity relies on a public key infrastructure (PKI) whose root of trust is represented by the Infrastructure Certification Authority that is managed and maintained by administrators.

The Device Inventory database plays an important role during the device authentication procedure, because it acts as a certificate whitelist that allows only registered certificates to be used during authentication. This feature has relevant security implications that are discussed in [Sections 4 and 5](#).

3.1. Discussion on related architectures

We discuss how the considered architecture relates to BeyondCorp ZTA ([Ward and Beyer, 2014](#)), NIST ZTA ([Rose et al., 2020](#)) and XACML architectures ([Ardagna et al., 2009](#); [XACML, 2013](#)).

The considered reference architecture is a subset of BeyondCorp ZTA (as already mentioned at the beginning of [Section 3](#)), because it focuses on preventive security measures, such as access control components that, if breached, give immediate unauthorized access to resources. BeyondCorp ZTA also includes additional supporting components such as security information and event management (SIEM) or threat intelligence systems. Our proposal is orthogonal to these supporting components and can be easily integrated with existing systems.

The NIST ZTA can be regarded as a full-fledged architecture that is similar to the BeyondCorp ZTA, however it adopts a higher abstraction level. The considered architecture represents a potential instantiation of the NIST ZTA: the Access Control Engine instantiates the Policy Decision Point (PDP), the Access Proxy instantiates the Policy Enforcement Point (PEP), the Access Policies database instantiates the Policy Administration Point (PAP), the User Group database instantiates

the Subject database, the Device Inventory database instantiates the Asset database, the Single Sign-on instantiates the ID Management System, and the Device and Infrastructure certification authorities instantiate the Enterprise PKI component. Finally, similarly to how our architecture is a subset of the BeyondCorp ZTA, it would also be a subset of the NIST ZTA because it only considers preventive security measures. Other supporting components could be easily integrated with our proposal.

Standard XACML architectures ([XACML, 2013](#)) focus on access control components but do not include many details regarding authentication. For access control, XACML architectures broadly share the same high level abstraction approach of the NIST ZTA, and many components also have the same names (e.g., PEP, PDP, PAP). Thus, our architecture can be also considered an instantiation of these components of XACML architectures. However, some components are even more abstract because they do not consider details related to distinguished authentication of users and devices (as an example, the Policy Information Point (PIP) store information that all ZTAs store in separated databases, such as the Subject database and the Asset inventory databases of the NIST ZTA). Finally, standard XACML architectures do not include other components, such as the Device Certification Authority and Single Sign-On system.

Some proposals extend XACML architectures to include identity management components to support credential-based access control ([Ardagna et al., 2009](#)). However, they consider a network agent that only includes the user identity and not his device, and thus lack the components devoted to device authentication.

4. Threat model and attack scenarios

We consider an attacker that wants to gain unauthorized access to some resource of an organization, such as users' information stored in databases or signing keys. The attacker may be an external threat, who does not possess a device registered in the Device Inventory and who is not registered in the User/Group database. The attacker may also be an insider threat, such as a disgruntled employee, who owns a managed device registered in the Device Inventory and who is registered in the User/Group database. Our proposal can defend against attackers that try violating Sources as well as any authentication and authorization component in order to compromise the integrity and authenticity of the access control process.

Attacks that gain access to Sources and other authentication and authorization components by violating the credentials of administrators or employees do not qualify as a component compromise because the attacker uses valid credentials. To prevent these attacks, our proposal can be combined with standard solutions for multi-factor authentication or decentralized governance approaches proposed by the literature ([Zhang et al., 2020](#)).

An attacker may try subverting the correct operations of zero trust access control by compromising the several components that, in the reference design of ZTAs, represent single points of vulnerability:

- **Device Certification Authority:** the attacker may compromise the confidentiality of the Device Certification Authority signing key. This allows the attacker to issue arbitrary certificates to attacker-controlled devices (A1). As the Device Inventory acts as a certificate whitelist, this attack does not allow the attacker to successfully authenticate his device because a corresponding entry that associates the rogue certificate to the attacker device must be present in the Device Inventory.
- **Infrastructure Certification Authority:** the attacker may compromise the confidentiality of the Infrastructure Certification Authority signing key. A similar compromise enables the attacker to issue rogue certificates to impersonate architectural components (A2). This attack becomes very effective in subverting zero trust access control if the attacker impersonates the Access Control Engine to the Access Proxy. In this case, the attacker can bypass the authorization logic by providing fake authorization results to the Access Proxy.
- **Device Inventory:** the attacker may compromise the integrity of the inventory by inserting arbitrary device entries (A3). This allows the attacker to insert an attacker-controlled device among the set of legitimate managed devices. If a similar operation is executed after the compromise of the Certification Authority, it allows the attacker to authenticate arbitrary devices (A1 + A3).
- **Access Policy Database:** the attacker may compromise the integrity of the access policies (A4), and grant access to unauthorized devices and users. If the attacker is an insider threat, this attack is sufficient to grant access to unauthorized resources as the insider device and user account are already registered in their respective databases. If the attacker is external, he can modify the policies to the extent that anonymous accesses are allowed.
- **Access Control Engine:** the attacker may compromise the integrity of the access control logic (A5) and alter the authorization logic to allow access to unauthorized devices and users.
- **Single Sign-On:** the attacker may compromise the confidentiality of the single sign-on signing key (A6). This allows the attacker to sign arbitrary authentication tokens, thereby impersonating registered users. If an insider compromises the Device Inventory to assign an attacker-controlled device to the impersonated user (A3 + A6), the attacker can access unauthorized resources. An external attacker must also compromise the certification authority to issue a valid certificate to the attacker-controlled device (A1 + A3 + A6).
- **User Group Database:** the attacker may compromise the integrity of the user group database (A7) to grant access to unauthorized users. If the attacker is an insider threat, this attack is sufficient to grant unauthorized access to resources as the insider may change his attributes to elevate his access privileges. An external attacker may leverage this attack to register valid credentials to the Single Sign-On, but he must still compromise the certification authority and the device inventory to issue a valid certificate to the attacker-controlled device (A1 + A3 + A7).
- **Access Proxy:** the attacker may violate the confidentiality of the secret keys used by the Access Proxy to authenticate

to resources (A8). Since the Access Proxy acts as a policy enforcement point, an attacker that is able to impersonate the Access Proxy can bypass any authentication and authorization logic.

We summarize the attacks against individual components and the cyber kill chains that allow unauthorized resource access in [Table 1](#).

The presented attacks allow to gain unauthorized access to cloud resources through different numbers and types of additional components that must be compromised. For example, compromising the Infrastructure Certification Authority (A2), the Access Policy database (A4), Access Control Engine (A5), the User Group database (A7) or the Access Proxy (A8) allows unauthorized access to resources because the attacker is able to alter or bypass the authorization logic without the need of compromising additional components. As a result, these attacks are very effective and require a minimum amount of successful compromises. We note that attacks A1, A3 and A6 do not grant unauthorized access on their own. In order to be effective, they must be included in a cyber kill chain consisting of multiple attacks.

If the attacker is unable to alter or bypass the authorization logic, then he must carry out attacks requiring the compromise of multiple components. In a similar way, the attacker can issue a request from an authorized device while impersonating an authorized user.

It is tempting to assume that compromising multiple architectural components is harder than compromising a single component and, as a consequence, that attacks involving the Device Certification Authority, the Single Sign-On system, the User Group database and the Device Inventory (A3 + A6, A1 + A3 + A7, A1 + A3 + A6) are unfeasible or unrealistic. This assumption is false when the architectural components are affected by the same vulnerabilities. For example, when they rely on the same software dependencies (e.g., same libraries), if they share the same operating system or if they run on the same hardware ([Kocher et al., 2019](#); [Lipp et al., 2018](#)). In practice this is a common occurrence as the additional costs of n-version programming ([Avizienis, 1995](#)) and of supporting heterogeneous hardware inhibit the creation of failure independent subsystems. In these scenarios, the efforts of compromising multiple components may not be more expensive than compromising one of them. As a result, designing survivable ZTAs for cloud computing environments that can resist to classes of seemingly complex attacks against multiple architectural components is of urgent and practical interest.

5. Zero trust architecture survivable design

The core zero trust principles ([Gilman and Barth, 2017](#)) that guide our design can be summarized as following:

- the network is always assumed to be hostile;
- external and internal threats exist on the network at all times;
- accesses from the local network do not guarantee trust;
- every device, user, and network flow must be authenticated and authorized;

Table 1 – ZTA attacks and kill chains summary.

Attack label	Attacked component(s)	Attack description	Unauthorized access
A1	Device CA	key confidentiality	✗
A2	Infrastructure CA	key confidentiality	external
A3	Device Inventory	data integrity	✗
A4	Access Policy database	data integrity	external
A5	Access Control Engine	process integrity	external
A6	Single Sign-On	key confidentiality	✗
A7	User Group database	data integrity	insider
A8	Access Proxy	key confidentiality	external
A3 + A6	Device Inventory Single Sign-On	kill chain	insider
A1 + A3 + A6	Certification Authority Device Inventory Single Sign-On	kill chain	external
A1 + A3 + A7	Certification Authority Device Inventory User Group database	kill chain	external

✗: attack does not grant unauthorized access.

These core principles are established to protect network resources, but we claim that they should also be extended to protect the ZTA itself from external and internal threats. Indeed, the multitude of attacks undermining the security of architectures presented in [Section 4](#) have the common characteristics of violating the trust of some components. The mitigation of the impact of trust violations due to successful compromise of trusted components requires the distribution of trust of each component among multiple entities. We identify two main approaches: *intra-component trust distribution* and *inter-component trust distribution*.

Intra-component trust distribution substitutes the original trusted component with a functionally equivalent structure composed by multiple replicas. Replicas share trust, so that the corruption of a subset of replicas is not sufficient to affect the security properties of the replicated component. Byzantine fault tolerant state machine replication is a well-known example of a similar approach ([Schneider, 1990](#)).

Inter-component trust distribution requires the distribution of trust among heterogeneous components sharing some dependency relation. This solution allows components to share trust on their dependencies with other components or actors. Whitelisting is an example of inter-component trust distribution: a component considers data received by a dependency as valid if it is both correct and approved. Data is correct if it satisfies certain semantics and formal properties. It is approved if it finds a match in the local whitelist. This component distributes trust because it allows the dependent component to discard seemingly valid but not approved data produced by a malicious dependency. To this aim, whitelisting relies on a trusted entity to define and update the whitelist. As a result, trust is shared between the dependency and the trusted entity: the dependency is trusted for correctness, while the entity defining the whitelist is trusted for approval. In our proposal, administrators represent the trust anchor of the ZTA. We can rely on them for whitelisting operations. Inter-component trust distribution based on whitelisting is already present in the reference ZTA in the form of the Device Inventory. It acts as a device certificate whitelist which

prevents dependent components (e.g., Access Control Engine and Access Proxy) from accepting forged device certificates that are correct but not approved. We rely on this observation to avoid the application of intra-component trust distribution techniques on the Device Certification Authority and the Infrastructure Certification Authority (see [Section 6.3](#)).

A so called *intermediary removal* is another example of inter-component trust distribution. Here, trusted dependencies are removed by shifting their responsibilities to the dependent components. This allows the system to distribute trust among all the components that depend on the removed intermediary because each component is independently responsible for guaranteeing the properties and functionality offered by the removed intermediary. In our proposal, we apply this technique by designing protocols that allow to move access control enforcement responsibilities from the Access Proxy to resources. This allows us to avoid the application of intra-component trust distribution techniques on the Access Proxy.

To sustain the workloads of modern cloud computing environments, the scalability and elasticity properties offered by cloud infrastructures are essential. Therefore, we consider the architectural components of our design as part of a cloud infrastructure owned and maintained by the infrastructure operator. Admins preserve direct access to the cloud infrastructure and, in accordance with the zero trust paradigm, our architecture allows users to access cloud resources both from within operator premises and remotely, provided they adopt authorized managed devices.

Our design of a survivable ZTA is guided by the threat model presented in [Section 4](#). For each attack, we identify the security properties that an attack may violate, and we derive the security requirements that the affected logical components must satisfy. Finally, for each affected component we identify the most appropriate trust distribution technique that can mitigate the identified attacks and protect the affected security properties. This approach preserves the logical components of the reference ZTA and their original functions, but it requires some additional components that operate together to guarantee the survivability of the architecture. The

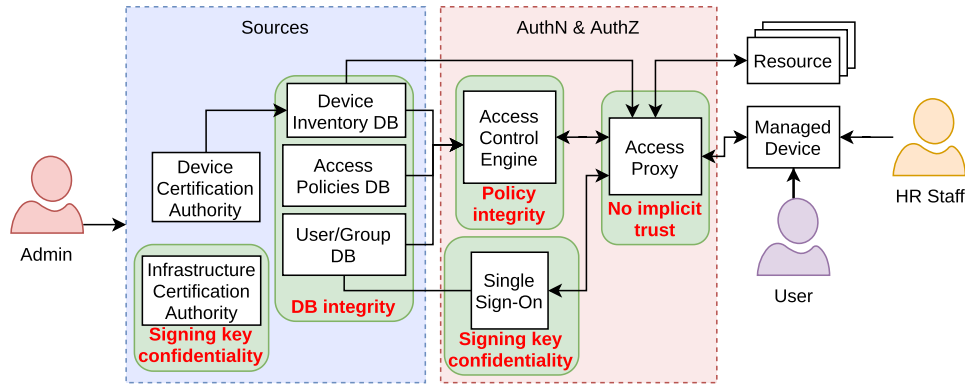


Fig. 2 – Security requirements that the components of a survivable zero trust architecture must satisfy.

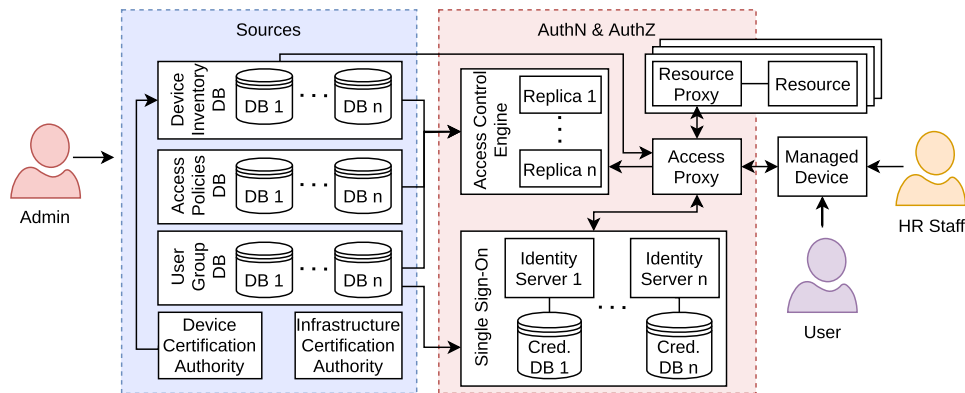


Fig. 3 – Overall survivable zero trust architecture.

main requirements that each component must satisfy are outlined in Fig. 2, while the resulting design is shown in Fig. 3.

Some attacks are possible because conventional databases do not tolerate malicious violations of the data integrity. As this attribute must be one of the main goals of a survivable architecture, we require that the Device Inventory, the User/Group database and the Policy Database can tolerate integrity violations by design and can recover from attacks. To this aim, we apply intra-component trust distribution and replicate the Device Inventory, the User/Group database and the Policy Database through Byzantine Fault Tolerant (BFT) database replication. These schemes distribute a database among a set of replicas that can mask malicious integrity violations and recover from them.

The Device Certification Authority is a single point of vulnerability because it is not designed to tolerate confidentiality violations of its signing key. An attacker that compromises the Device Certification Authority can issue rogue certificates to attacker-controlled devices that can later try to authenticate by presenting a valid certificate. However, a BFT Device Inventory is sufficient to prevent this attack. In fact, the Device Inventory acts as a certificate whitelist that prevents authentication with unregistered certificates. Since the attacker is unable to maliciously alter a BFT Device Inventory to insert rogue certificates, he is unable to authenticate his rogue devices. Our design leverages a similar whitelist-based inter-

component trust distribution, and does not require a Device Certification Authority that can tolerate the violation of signing key confidentiality because the BFT Device Inventory already masks this attack.

The Infrastructure Certification Authority is a single point of vulnerability as an attacker that compromises its signing key can issue a rogue certificate to impersonate infrastructure components. In order to mitigate this attack, we apply whitelist-based trust distribution on the Infrastructure Certification Authority that allows dependent components to detect forged certificates not belonging to the component local certificate whitelist. As a result, our design can securely adopt conventional certification authorities that rely on well-established security techniques to detect and recover from key compromise.

The type of user impersonation attacks violates the confidentiality of the signing key that is used to issue single sign-on authentication tokens to users. In such a way, an attacker is able to forge rogue authentication tokens. For this reason, we require single sign-on authentication mechanisms that can tolerate and recover from attacks that compromise the confidentiality of the signing key. To guarantee signing key confidentiality, we apply intra-component trust distribution by adopting a survivable single sign-on mechanism. This mechanism splits the signing key among multiple signing parties so that, if a subset of parties is compromised, then the confiden-

tiality of the signing key is guaranteed; moreover, the compromised parties can recover from an attack. We allow each signing party to access the Users/Group database to include user information in the issued authentication tokens.

The Access Control Engine represents a single point of vulnerability because an adversary that can compromise the integrity of its access control logic may be able to gain unauthorized access to cloud resources. To guarantee integrity of the access control logic we apply intra-component trust distribution by replicating the Access Control Engine through BFT schemes so that it can tolerate the compromise of a threshold of its replicas.

Finally, the Access Proxy is another single point of vulnerability because an adversary breaching it can violate the confidentiality of its keys used to establish authenticated communication channels with cloud resources. This vulnerability allows the attacker to access resources by impersonating the Access Proxy and bypass any authentication and authorization logic. This is due to the implicit trust that each resource grants to the Access Proxy. To fix a similar vulnerability we notice that the Access Proxy has many responsibilities that do not have to remain concentrated in a single component. In particular, the Access Proxy is responsible for both device authentication and policy enforcement. Device authentication can be left to the Access Proxy, while policy enforcement can be decentralized. To this aim, we apply inter-component trust distribution based on intermediary removal by extending each resource with a Resource Proxy, also known as *Zero Trust reverse proxy* (Gilman and Barth, 2017) or *gateway* (Rose et al., 2020). Each resource trusts its Resource Proxy to enforce access control. This design approach distributes trust among multiple Resource Proxies instead of concentrating it in a single Access Proxy. The advantage is twofold: a compromised Access Proxy does not allow an adversary to access resources; a compromised Resource Proxy does not allow an adversary to access other resources managed by other non-compromised Resource Proxies, thus limiting the impact of the compromise of a trusted component.

6. Detailed architecture

We describe how the trust distribution techniques of the architectural design are applied to each component. Section 6.1 analyzes survivable databases used in multiple components of the architecture. Section 6.2 discusses the survivable single sign-on component. Section 6.3 discusses trust distribution for certification authorities. Section 6.4 describes the novel design of the access control engine and of the access proxy.

6.1. Survivable databases

Survivable databases can be obtained by means of middleware-based *byzantine fault tolerant (BFT) database replication* (also, BFT databases), which are full-fledged database management systems (DBMS) that achieve BFT guarantees. Although the initial BFT databases date back to some decades ago (Molina et al., 1986), they have been improved by more

recent research proposals (e.g., Garcia et al., 2011; Gashi et al., 2004, 2007; Luiz et al., 2014; Pedone and Schiper, 2012; Vandiver et al., 2007). We show that no existing BFT database can satisfy all the requirements of our survivable ZTA.

Table 2 summarizes the analysis of existing BFT databases. In this table, we consider typical features (Luiz et al., 2011), such as *number of replicas*, *message complexity*, *consistency guarantees*, *support of concurrent transactions* and *tolerance of faulty clients*. Moreover, we introduce additional requirements that are inherited by the literature on survivable systems (Avizienis et al., 2004): *presence of single points of vulnerability* and *fault recovery*.

Single points of vulnerability. BFT database replication solutions may introduce single points of vulnerability (denoted as “SPoV” in Table 2) in their architecture as a compromise between survivability and performance.

Number of replicas. The number of replicas that a BFT database requires corresponds to the number of distinct database instances that must be deployed to guarantee byzantine fault tolerance. Most proposals require a number of $n = 3f + 1$ replicas, where f is the number of faulty replicas that the system can tolerate. For example, we need four replicas to tolerate $f = 1$ faults.

Fault recovery. This property guarantees that the database can recover from a compromise of some of its replicas. Moreover, it ensures that the number of faulty replicas f does not exceed the amount established at system setup during the whole lifetime of the database.

Message complexity. The message complexity indicates the growth of the number of messages that are globally exchanged among replicas during the protocol execution with respect to the number of replicas. Most proposals require a quadratic number of messages in the number of replicas.

Consistency guarantees. The consistency guarantees indicate the tolerance of the replicated database to anomalies arising from the concurrent execution of transactions. Single-copy serializability (denoted as “serializable” in Table 2) guarantees that the execution of concurrent transactions on a replicated database behaves as the sequential execution of the same transactions on a single copy of the database (Bernstein and Goodman, 1985). This solution represents the strongest consistency guarantee for replicated databases. On the other hand, snapshot isolation is a weaker consistency guarantee that may affect data integrity because it does not prevent some classes of anomalies (Berenson et al., 1995). Other proposals adopt a similar isolation level because, especially for read biased workloads, it offers a better transaction throughput. In our case, single-copy serializability is preferable to snapshot isolation because an attacker may exploit anomalies to maliciously corrupt data.

Concurrent transactions. A database may not support the execution of concurrent transactions, such as in the case of Gashi et al. (2004). This is a limitation that may negatively impact performance in scenarios characterized by intensive database workloads.

Faulty clients. A BFT database tolerates byzantine clients if it does not allow a client to compromise the database consistency guarantees during transaction execution. As an example, a byzantine client may equivocate responses to different

Table 2 – Comparison among BFT database replication proposals.

	SPoV	# replicas	Fault recovery	Message complexity	Consistency	Concurrent transactions	Faulty clients
HRDB	yes	$2f + 1$	yes	$O(n)$	serializable	yes	yes
Gashi et al.	no	$3f + 1$	yes	$O(n^2)$	serializable	no	yes
BFT-DU	no	$3f + 1$	no	$O(n^2)$	serializable	yes	no
Byzantium	no	$3f + 1$	no	$O(n^2)$	snapshot	yes	yes
MITRA	no	$3f + 1$	no	$O(n^2)$	serializable	yes	yes

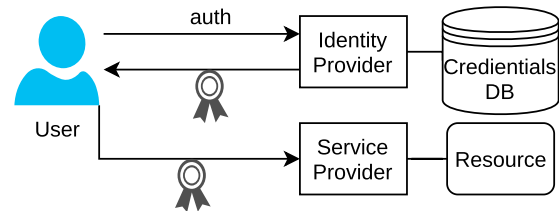
replicas during protocol execution to bring the system in an inconsistent state.

The evaluation criteria allow us to identify the requirements that a BFT database should satisfy in a survivable ZTA. A BFT database should not introduce single points of failure. It should ensure single-copy serializability and allow concurrent transactions. It should tolerate byzantine clients and guarantee fault recovery. Existing proposals only partially satisfy all these requirements. Hence, further research efforts are required in the field of BFT databases. We identify MITRA and HRDB as the solutions offering the best trade-offs among the examined BFT databases. MITRA satisfies most requirements but it does not include a fault recovery mechanism. Hence, if faults can be detected, then the database may be restored to a known safe checkpoint, but non-detectable byzantine faults may go unnoticed and accumulate over time among replicas so that the security threshold is exceeded (Gashi et al., 2007). HRDB supports fault recovery and offers better performance than MITRA (Section 8) at the cost of introducing a trusted component in its design. The trusted component has a much lower attack surface compared to database replicas as it can be implemented with a smaller code base (Vandiver et al., 2007). The adoption of secure coding and operations best practices to protect the trusted component should guarantee that HRDB can be regarded as a practical proposal in the context of survivable ZTAs.

It is interesting to evaluate some architectural features of existing BFT databases that determine how they may be integrated in a survivable ZTA. A common approach to BFT databases is to design a middleware that enables crash-fault tolerant databases to collectively guarantee a BFT service. The middleware intercepts client requests and delivers them to the appropriate replicas, thereby hiding BFT replication to the client. This black box approach has several advantages: it does not require modifications of the database source code, which may not be available in case of proprietary databases. Moreover, it allows the inclusion of several databases from different vendors in the deployment of the BFT system. This feature is important because it allows middleware solutions to support *design diversity* (Avizienis, 1995), which is an important security technique to mitigate common-mode failures.

In general, the architecture of a middleware-based BFT database consists of a client-side proxy component that is aware of each replica in the system. The proxy intercepts the client requests and appropriately forwards them to the replica-side proxy which then communicates with the replica's database management system (DBMS).

Our design replaces the User/Group and the Access Policies databases with a different BFT database, and may replace the Device Inventory to obtain stronger survivability guaran-

**Fig. 4 – Traditional SSO architecture and flow.**

tees (for a precise definition of survivability guarantees see Definition 7.1). To this aim, we must establish the desired level of tolerable byzantine replicas f for each BFT database, and then deploy the required total number of replicas. Moreover, the single sign-on system, the Access Control Engine and the Access Proxy, which are clients of the BFT databases, must be modified to include the client-side proxy.

6.2. Survivable single sign-on

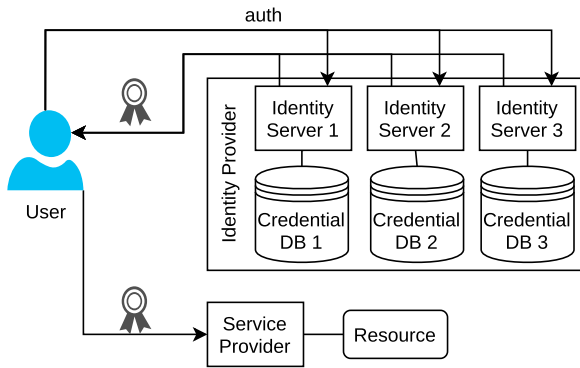
A survivable single sign-on function requires a cloud-based *survivable single sign-on* architecture. The examples in Agrawal et al. (2018); Baum et al. (2020); Zhang et al. (2020) are some single sign-on systems that guarantee signing key confidentiality in case of successful attacks and allow recovery from successful compromises. We establish the most appropriate system that our architecture can adopt, by analyzing the state-of-the-art about traditional and survivable single sign-on solutions and identify the characteristics that are required for survivable ZTAs.

Traditional single sign-on, represented in Fig. 4, involves three actors: a user, and identity provider and a service provider. The user wants to access a resource owned by the service provider, who enforces access control over the resource. As a result, the service provider needs to authenticate users. To this aim, the service provider trusts and delegates to the identity provider the authentication procedure. The service provider does not maintain the user credentials database required for authentication, which is maintained by the identity provider. A user, who has previously registered himself to the identity provider, can use his credentials to authenticate to the identity provider. Upon successful authentication, the identity provider digitally signs and sends an authentication token to the user. The user then forwards the authentication token to the service provider. The service provider verifies the token authenticity and, if authentic, can be convinced about the user identity.

Various attacks against traditional single sign-on systems motivate the need for survivable single sign-on. An at-

Table 3 – Comparison among survivable single sign-on proposals.

	Threshold	Proactive	Trusted setup	# auth. round trips	OPTA	# secrets per replica
PASTA	(t, n)	no	no	1	no	$O(\#users)$
PESTO	$(n - 1, n)$	yes	yes	2	yes	$O(n)$
PROTECT	(t, n)	yes	no	2	yes	$O(\#users)$

**Fig. 5 – Survivable SSO architecture and flow.**

tacker that can compromise the confidentiality of the identity provider signing key is able to forge arbitrary authentication tokens. Moreover, an attacker that can read the credentials database may be able to mount offline dictionary attacks. A survivable single sign-on system, represented in Fig. 5, prevents these attacks by substituting the identity provider with set of n identity servers, each maintaining independent credentials databases. To prevent an adversary from forging arbitrary authentication tokens, the identity provider signing key is cryptographically split among n identity servers so that any subset of less than t servers cannot reconstruct or use the signing key. This approach guarantees *token unforgeability* against the breach of $t - 1 < n$ identity servers. To prevent an attacker from mounting offline dictionary attacks, user credentials are cryptographically split among multiple credentials databases so that gathering data stored in less than t databases does not reveal any information about user credentials. This guarantees *credentials security* against the breach of $t - 1 < n$ credentials databases.

The information flow of survivable single sign-on is similar to the flow of traditional single sign-on. In survivable single sign-on, the user authenticates to a threshold of identity servers which issue enough partial authentication tokens so that the client can reconstruct a full authentication token. The client then forwards the authentication token to the service provider as in traditional single sign-on.

We identify the characteristics of survivable single sign-on (SSO) and establish the requirements that survivable SSO must satisfy to be adopted in a survivable ZTA. To this aim, we consider typical features adopted in the survivable single sign-on research field (Baum et al., 2020), that are: *threshold security*, *proactive recovery support*, *trusted setup*, *number of round trips for authentication*, *resistance against online password testing attacks*, *number of secret keys per replica*. Table 3 compares the considered proposals.

Threshold security. This criterion indicates whether the SSO system allows the choice of any threshold $t < n$ or imposes additional constraints on the value of t . In the case of PESTO (Baum et al., 2020), the value of t can be set just to $t = n - 1$. This implies that a token generated by this proposal is valid only if it is authenticated by all identity servers. Therefore, if a single identity server becomes unresponsive, the whole SSO becomes unavailable as no valid authentication tokens can be generated. As a result, allowing the flexible definition of the threshold t , such as in solutions that guarantee (t, n) threshold security, can guarantee a higher level of availability of the single sign-on service than solutions that guarantee $(n - 1, n)$ threshold security.

Proactive security. A survivable SSO proposal is proactively secure if it allows to periodically refresh the cryptographic material of identity servers. This allows us to recover compromised identity servers from successful attacks and ensures that any leaked cryptographic material becomes useless after a refresh. This property is essential to guarantee that the amount of compromised identity servers never exceeds the security threshold t during the whole lifetime of the single sign-on service.

Trusted setup. A survivable SSO may require a trusted setup to split cryptographic material among identity servers. A trusted setup introduces a possible single point of failure in the distributed architecture. However, in an enterprise scenario a trusted setup may be acceptable in case it is executed by an administrator who must be trusted anyway because, for example, he needs direct access the identity servers for setup and maintenance purposes. Moreover, schemes that rely on trusted setup usually are more efficient than alternative schemes not relying on trusted setup. As an example, PESTO, which relies on a trusted setup, allows a non-interactive proactive refresh phase. PROTECT, which does not rely on a trusted setup, incurs in a quadratic communication cost among servers during the proactive refresh procedure.

Number of round trips for authentication. It indicates the number of round trips of communication between a client and each identity server during authentication. Current proposals either require one or two round trips of communication. While reducing the number of round trips from two to one clearly benefits communication efficiency, research shows that this opens the possibility of online password testing attacks (OPTA) (Baum et al., 2020; Zhang et al., 2020), which we describe shortly. Determining whether it is possible to reduce the number of rounds to one while preventing OPTAs is still an open problem.

Online password testing attacks. This criterion indicates whether an attacker is able to perform an online password testing attack. This attack allows an adversary with a dictionary of candidate passwords to retrieve the identity of users

using a particular password in the dictionary. A desirable property of a survivable single sign-on system is to prevent these attacks as this allows weaker assumptions on the strength of user credentials.

Number of secrets per replica. It indicates the growth of the amount of secret cryptographic material as a function of some parameters, such as the number n of identity servers or the number of users (denoted as “#users” in Table 3). In PESTO, user-specific information stored in each credential database is not confidential and as a result, the amount of confidential cryptographic material does not depend on the number of users that we assume to be much larger than the number of identity servers. On the other hand, PASTA and PROTECT register users by storing security-sensitive cryptographic material in the credentials database and thus the amount of confidential cryptographic material depends on the number of users. A low amount of confidential cryptographic material may allow the identity servers to securely store such material in a hardware security module enhancing the overall security.

Let us identify the main requirements that a survivable SSO system must satisfy to be adopted in a survivable ZTA. First of all, it must guarantee proactive security and prevent online password testing attacks. Moreover, it must guarantee (t, n) threshold security to provide higher availability with respect to $(n - 1, n)$ threshold security. To the best of our knowledge, PROTECT is the only proposal satisfying all previous requirements.

To integrate the survivable SSO in our design, we assume that the Users/Group is a middleware BFT database. The survivable SSO system must have access to the Users/Group database to include user identity information in authentication tokens. This implies that each identity server must be modified to include the client-side proxy, as discussed in Section 6.1. There are different design decisions that may be considered as appealing optimizations, but some of them are useless or even harmful in terms of security. The two not recommended optimizations include the replication of the credentials databases of each identity server, and the memorization of all user credentials in one database. One may hope that replicating the credentials database of each identity server through BFT database replication can increase the security of the SSO system, but this is not true. Credentials database distribution in the context of survivable SSO enhances security by cryptographically splitting credentials at multiple locations to prevent offline dictionary attacks. On the other hand, BFT replication in the context of databases enhances data integrity and availability. Moreover, it prevents equivocation in presence of byzantine adversaries. These two security goals are distinct, hence replicating a credentials database does not increase the difficulty of offline dictionary attacks. In summary, replicating credential databases is useless in terms of security, and possibly harmful in terms of performance due to the overhead of BFT replication.

The second not recommended optimization involves the reduction of the number of credentials databases by storing user credentials in the Users/Group database. However, survivable SSO systems are designed with the assumption that credentials databases are distinct for each identity server and that breaches are independent. Storing credentials in the same database, even if it is BFT-replicated, vi-

olates this fundamental assumption as it allows an adversary who has breached the Users/Group database, or any of its replicas, to collect all credentials at once, thus effectively violating more than t identity servers. Therefore, reducing the number of credentials databases is a bad design decision that harms the security of SSO and that must be avoided.

6.3. Certification authorities

We detail the trust distribution techniques applied to the Device Certification Authority and the Infrastructure Certification Authority of the reference ZTA. This latter CA is a single point of vulnerability that, if breached, allows an attacker to issue valid certificates to attacker-controlled devices and to impersonate other components of the ZTA. A possible solution is to continue the trend of intra-component trust distribution proposed in Sections 6.1 and 6.2 towards the adoption of survivable certification authorities (Reiter et al., 1996; Wu et al., 1999; Zhou et al., 2002). This is an unnecessary step because we show that applying inter-component trust distribution and maintaining conventional centralized certification authorities guarantees enough security against forged certificates.

Let us first consider the Device Certification Authority. We assume that the Device Inventory is BFT-replicated (Section 6.1) and, as a consequence, the attacker is unable to compromise its integrity. Hence, even if an attacker compromises the confidentiality of the Device Certification Authority signing key, the resulting rogue certificates issued to attacker-controlled devices are useless to the attacker because the Device Inventory acts as a certificate whitelist. Thus, even if an attacker issues a rogue certificate for a device that the attacker owns, the certificate does not allow the attacker to satisfy existing access control policies because the attacker device is not present in the Device Inventory. As a result, during device authentication (Section 3), the Access Proxy denies the authentication to the attacker device as its rogue certificate is not present in the Device Inventory. The only way for an attacker to effectively use a rogue certificate is to add it to the Device Inventory. As the Device Inventory is BFT replicated, the attacker cannot proceed in a similar way.

We now focus on the Infrastructure Certification Authority. The effect of attacks in which the adversary issues rogue certificates to impersonate architectural components can be mitigated by means of key pinning (Clark and Oorschot, 2013). In fact, key pinning is a form of certificate whitelisting. In key pinning, administrators configure the several architectural components to communicate over secure mutually authenticated communication channels which adopt admin-installed certificates to authenticate the endpoints. As a result, even if the attacker issues a rogue certificate and uses it in an impersonation attempt, the attacked components refuse to connect to the attacker because the rogue certificate is different from the component's pinned certificate.

Our design adopts well-established security techniques to detect and recover from key compromise of centralized certification authorities. Moreover, our design relies on a scalable cloud infrastructure to ensure a high level of availability of the certification authority service to sustain the workloads of modern cloud computing environments.

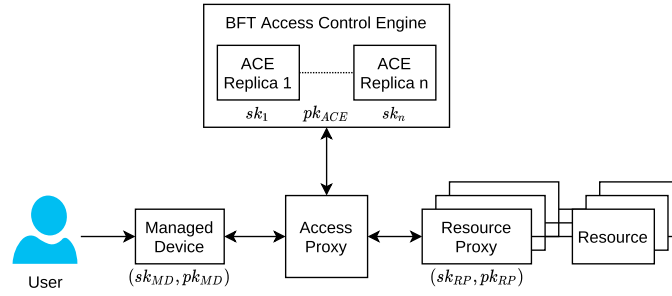


Fig. 6 – BFT access control engine and access proxy architecture proposal.

6.4. Access control engine and access proxy

To mitigate attacks against a compromised Access Control Engine, we apply intra-component trust distribution to the Access Control Engine. Moreover, to mitigate the attacks against a compromised Access Proxy, we apply inter-component trust distribution based on intermediary removal, where we remove the responsibility of access control enforcement from the Access Proxy. To securely integrate the two design choices we deploy a secure communication protocol that guarantees end-to-end authenticity of user requests and Access Control Engine policy evaluations, and end-to-end confidentiality of resource responses.

We describe the details of our proposal by considering Fig. 6. Our proposal extends resources with a Resource Proxy, which enforces access control policies on its corresponding resource. It accepts requests from the Access Proxy and forwards them to the resource only if they are collectively authenticated by a threshold of Access Control Engine replicas, thus preventing unauthorized access to the resource by a compromised Access Proxy. If we assume that an attacker cannot compromise more than a threshold of Access Control Engine replicas, then the attacker is unable to forge an Access Control Engine collective signature.

The high-level information flow of the proposed protocol is as follows. The user submits a request to the Access Proxy and completes both *device authentication* and *user authentication* as in the reference flow. If user and device authentication are successful, the Access Proxy sends the authenticated user request to the Access Control Engine replicas which evaluate the access policies. Replicas agree on the outcome of the policy evaluation and return a collective signature of the request to the Access Proxy if access is allowed. The Access Proxy forwards the signed request to the Resource Proxy which verifies the request signature and, if valid, the Resource Proxy forwards the request to the resource. The resource sends the response to the Resource Proxy which encrypts it for the managed device and forwards the encrypted response to the Access Proxy. The Access Proxy then returns the encrypted response back to the managed device which decrypts it to obtain the plaintext response. We highlight that our proposal does not protect request confidentiality because it may have negative impact on system performance and, perhaps counterintuitively, even on security (we propose a discussion in the last paragraph of this section).

We now discuss how the proposed architecture can be deployed adopting existing solutions. The Access Control Engine can be decentralized by adopting *proactive byzantine fault tolerant* protocols, such as [Castro and Liskov \(2002\)](#) by deploying n replicas. This protocol allows to achieve *byzantine fault tolerant state machine replication* ([Schneider, 1990](#)) while also guaranteeing recovery from a compromise of $t < n/3$ replicas. Each replica of the Access Control Engine requires access to the Access Policies, Users/Group and Device Inventory databases to determine the outcome of the access control logic. Moreover, the Access Proxy requires access to the Device Inventory database to access whitelisted device certificates during device authentication. As a result, both Access Control Engine and Access Proxy replicas must integrate the client-side proxy of the adopted middleware-based BFT database replication solution discussed in [Section 6.1](#). In our proposal, we assume that the setup of a proactively secure threshold signature scheme, such as [Boldyreva \(2003\)](#), is completed so that each replica holds the secret share sk_i of the secret key corresponding to the replicas collective public key pk_{ACE} . The security threshold of the signature scheme must match the threshold t of the BFT protocol. We assume that the Resource Proxy knows the Access Control Engine collective public key pk_{ACE} . Moreover, we assume that the Resource Proxy owns a key pair (sk_{RP}, pk_{RP}) and that the managed device knows the public key pk_{RP} . In the following, we denote as (sk_{MD}, pk_{MD}) the public key pair of the user managed device, whose certificate is stored in the Device Inventory. Finally, we assume that communication among components occurs over confidential and mutually authenticated channels. We note that during proactive refresh, the cryptographic material of both the proactive BFT protocol and of the threshold signature scheme are refreshed.

We assume that the goal of the attacker is to gain unauthorized access to resources. To this aim, the attacker can compromise $t < n/3$ Access Control Engine replicas. The attacker can also compromise the Access Proxy to eavesdrop and modify all messages in transit and send new messages authenticated with the Access Proxy cryptographic material. Finally, we assume a computationally bound adversary that cannot break the security of the adopted cryptographic schemes.

We describe the detailed information flow and summarize it in [Fig. 7](#). The user generates the request, which we denote as req , to access a resource. The user managed device then computes the pair (req, σ_{MD}) , where σ_{MD} denotes the signature on req produced with sk_{MD} . The user completes

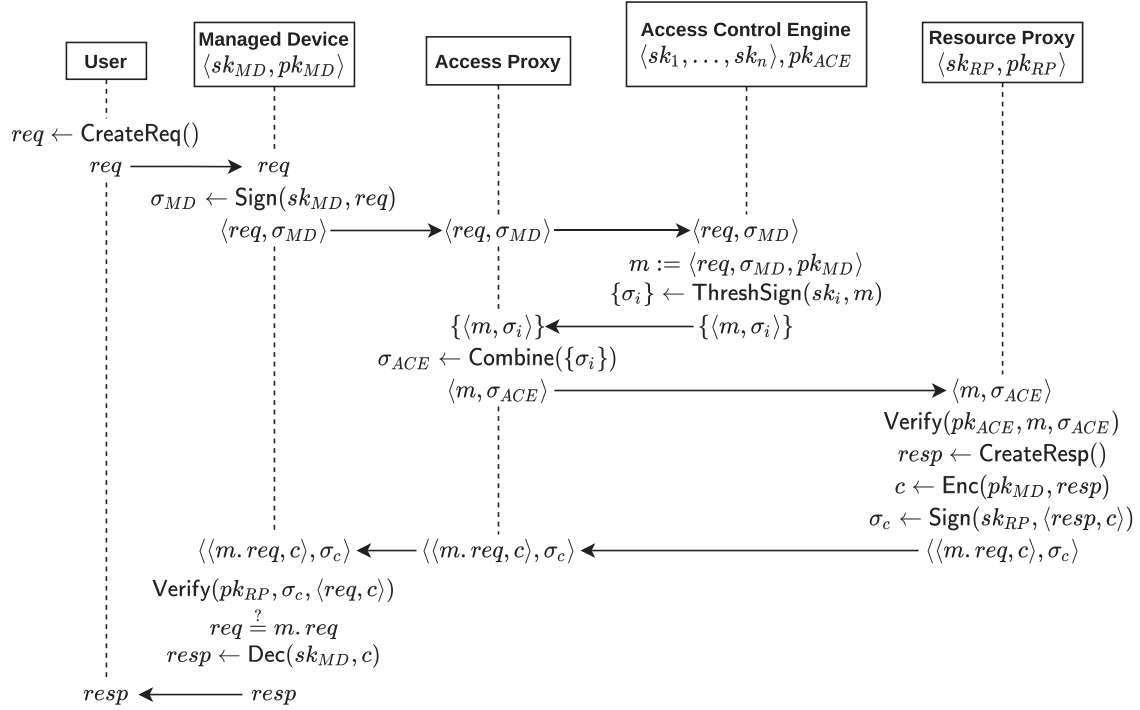


Fig. 7 – Resource access protocol flow.

the user authentication and device authentication procedures (see Section 3) and sends $\langle req, \sigma_{MD} \rangle$ to the Access Proxy. We assume that req contains the due information required by the Access Control Engine BFT protocol, including metadata to defend against typical adversarial attacks (as an example, a nonce to prevent replay attacks). The Access Proxy sends $\langle req, \sigma_{MD} \rangle$ along with the authentication information obtained during device and user authentication to the Access Control Engine replicas. Each replica verifies the user and device authentication information as well as σ_{MD} . Each replica, given the authenticated request req , evaluates the access control policies. If access is granted, then replicas collectively sign an authorization message. To this aim, each replica computes $\langle \langle req, \sigma_{MD}, pk_{MD} \rangle, \sigma_i \rangle$ where σ_i denotes a signature share produced by replica i on message $\langle req, \sigma_{MD}, pk_{MD} \rangle$. We note that each replica has access to the Device Inventory so they can all obtain the same pk_{MD} . The Access Proxy collects at least $t + 1$ responses and then combines signature shares σ_i into a signature σ_{ACE} which can be verified with pk_{ACE} . The Access Proxy then sends $\langle \langle req, \sigma_{MD}, pk_{MD} \rangle, \sigma_{ACE} \rangle$ to the Resource Proxy. The Resource Proxy verifies σ_{ACE} with pk_{ACE} and, if the signature is valid, forwards req to the resource. The resource, given request req , computes the response $resp$ and then sends $resp$ back to the Resource Proxy. The Resource Proxy encrypts $resp$ with the managed device public key pk_{MD} , producing ciphertext c , and finally signs the pair $\langle req, c \rangle$ with his secret key sk_{RP} , producing signature σ_c . The Resource Proxy then sends $\langle \langle req, c \rangle, \sigma_c \rangle$ to the Access Proxy, which simply forwards it to the managed device. The managed device finally verifies that req matches the request sent initially, verifies σ_c with the Resource Proxy public key pk_{RP} and then decrypts c using his secret key sk_{MD} .

We now discuss the security of our proposal. The attacker is not able to alter the request req sent by the user because it is authenticated by the user secret key, which is itself authenticated by the corresponding certificate stored in the Device Inventory. The attacker cannot replay the request to replicas because we assumed that req already embeds information to prevent replay attacks. The attacker is unable to forge an Access Control Engine collective signature as we assume that the attacker cannot compromise more than a threshold of replicas to gather enough key shares sk_i to produce arbitrary signatures. As a result, even if the attacker compromises the Access Proxy, it cannot gain direct access to resources. The attacker is unable to forge or replay old responses to the managed device because it is unable to forge Resource Proxy signatures which authenticate ciphertexts. Finally, the attacker is unable to eavesdrop on resource responses because they are encrypted for the managed device.

The availability guarantees of the proposed solution against denial of service (DoS) attacks involve some trade-offs that are worth discussing. We consider DoS attacks against the Access Proxy because it is a client-facing component and thus more likely to suffer from DoS. Moreover, we also consider DoS attacks originating from a compromised Access Proxy that silently drops user requests. In the proposed protocol Access Control Engine replicas collectively sign an authorization message only if access is granted. When access is denied to the user, Access Control Engine replicas do not respond with a collectively signed error message. The lack of response prevents the user from distinguishing an available ZTA that is denying access from an unavailable ZTA under DoS. To allow the user to detect DoS attacks, the protocol may be modified to also let

Access Control Engine replicas collectively sign access-denied error messages. However, we note that collective signing is an expensive operation in terms of computational and network costs and thus signing every user request even when access is denied may facilitate DoS attacks because of the increased load on the Access Control Engine. Moreover, we note that the lack of response messages may actually be a desired behavior for a ZTA. In fact, Single Packet Authorization (SPA) is a technique that may be adopted to avoid fingerprinting of the infrastructure and resources. With SPA the Access Proxy only accepts ingress packets authenticated by known hosts, and silently drops packets from unknown hosts to conceal its presence (Gilman and Barth, 2017). If SPA is adopted, then revoked access at the SPA level is indistinguishable from an unavailable ZTA under DoS. Therefore, collective signing of access-denied error messages involves trade-offs that should be evaluated depending on the specific scenario that adopts the proposed architecture.

The security of the Resource Proxy component also deserves further consideration. In the proposed design the resource trusts its Resource Proxy for policy enforcement. Designing a survivable Resource Proxy would certainly enhance the security guarantees of the architecture. However, we conjecture that components responsible for access control enforcement must be trusted by dependent components (resources in our case). Our conjecture is supported by the fact that many related intrusion tolerant architectures independently obtain designs similar to ours by adopting trusted components for access control enforcement (Dhar and Bose, 2020; Garcia et al., 2016; Maesa et al., 2017, 2018, 2019). Designing a survivable enforcement point is an interesting challenge for future work.

As anticipated at the beginning of this section, we note that the attacker is still able to eavesdrop on user requests. This may allow the attacker to infer enough information about the requested resource to compromise its confidentiality. However, we claim that protecting request confidentiality may have negative impact on system performance and on security. The confidentiality of the request must be protected at multiple points in the proposed architecture: it must be protected from a compromised the Access Proxy and it must be protected from any compromised Access Control Engine replica. To motivate the negative impact on system performance, we point out that to allow Access Control Engine replicas to evaluate policies while protecting request confidentiality, we must adopt *universally verifiable multiparty computation* schemes, such as Schoenmakers and Veeningen (2015). These schemes allow a set of parties (the Access Control Engine replicas) to evaluate a function over secret inputs (provided by the user) and produce a result whose correctness can be publicly verified by the Resource Proxy which did not participate in the computation. These schemes are far from being practical and would impose a severe overhead on policy evaluation. Moreover, they have a negative impact on security because confidential requests prevent external tools from providing contextual threat information to the Access Control Engine, such as in the case of intrusion detection systems (Apruzzese et al., 2017). Therefore, guaranteeing confidentiality of requests and contextual request information may be conflicting requirements.

7. Security analysis

In this section we analyze the overall security of the proposed survivable ZTA.

In the reference threat model we identify sequences of attacks that allow the adversary to gain unauthorized access to a resource. These attacks are also relevant to the proposed survivable ZTA. The difference is that attacks against a survivable component must compromise an amount of replicas that exceeds the component security threshold.

To quantitatively assess the effectiveness of our design choices we introduce novel definitions of survivability in the context of ZTAs.

Definition 7.1 (Zero trust survivability level). The survivability level of the zero trust architecture is the minimum amount of replicas that an attacker must compromise to gain unauthorized access to a resource in any attack sequence.

Intuitively, we say that a ZTA is survivable if there is no attack sequence that allows an adversary to gain unauthorized access to a resource by compromising a single replica. We formalize this intuition in Definition 7.2.

Definition 7.2 (Survivable ZTA). A zero trust architecture is survivable if its survivability level is greater than one.

Given Definitions 7.1 and 7.2 and the attack sequences that apply to the proposed architecture, we can derive a meaningful criterion to assign values to the security threshold of each replicated component to determine the amount of its replicas. We consider components that are not replicated as a single replica. We define the security thresholds of replicated components as follows:

- f_{DIdb} : device inventory database
- f_{APdb} : access policy database
- f_{UGdb} : user group database
- f_{ACE} : access control engine
- f_{SSO} : single sign-on

In Table 4 we report the costs of the identified attack sequences in terms of number of replicas that the attacker must compromise.

We note that attack sequences that include attacks A1 involve the compromise of the certification authority, which is not a replicated component and therefore counts as a single replica. We also note that the above costs are reported considering the best case scenario in which all components and replicas are failure independent. As already discussed in Section 4, it is essential to guarantee failure independence among components (inter-component independence) and within components replicas (intra-component independence). Otherwise, all components and replicas that share common mode failures count as a single replica in evaluating the complexity of attack sequences.

Attacks A4, A5, A1 + A3 + A6 and A1 + A3 + A7 allow an external adversary to access a resource, whereas attacks A7 and A3+A6 allows only an internal adversary to gain unauthorized access to a resource. As shown in Section 6.4, the proposed solution prevents attack A8 because an adversary that com-

Table 4 – Attack costs.

Attack sequence	Attacked component(s)	Attack cost	Attacker type
A1	Device CA	–	–
A2	Infrastructure CA	X	external
A3	Device Inventory database	–	–
A4	Access Policy database	$f_{APdb} + 1$	external
A5	Access Control Engine	$f_{ACE} + 1$	external
A6	Single Sign-On	–	–
A7	User Group database	$f_{UGdb} + 1$	internal
A8	Access Proxy	X	external
A3 + A6	Device Inventory	$f_{Dldb} + f_{SSO} + 2$	internal
	Single Sign-On		
A1 + A3 + A6	Certification Authority	$f_{Dldb} + f_{SSO} + 3$	external
	Device Inventory		
	Single Sign-On		
A1 + A3 + A7	Certification Authority	$f_{Dldb} + f_{UGdb} + 3$	external
	Device Inventory		
	User Group database		

X: prevented attack. –: attack not effective even in reference ZTA.

promises the Access Proxy is not able to gain unauthorized access to resources. Moreover, attack A2 is also prevented by certificate whitelisting.

The costs of the attacks highlight the importance of adopting a survivable Access Policy database, a survivable User Group database and survivable Access Control Engine. Adopting non-survivable alternatives ($f_{APdb} = 0$ and $f_{ACE} = 0$ respectively) does not allow to design a survivable ZTA that satisfies the proposed survivability definition. The costs of attack sequences A1 + A3 + A6, A1 + A3 + A7 and A3 + A6 highlight possible trade-offs in terms of component replication during a real deployment. It is sufficient to guarantee that the cost of these attack sequences is greater than one to obtain a survivable ZTA. As an example, to achieve the minimum survivability level it is sufficient to guarantee that the Device Inventory and Single Sign-On components are failure independent, while it is necessary to adopt a survivable Access Policy database, a survivable User Group database and a survivable Access Control Engine. To achieve higher survivability levels it is required to adopt the intra-component trust distribution techniques also for the Device Inventory and Single Sign-On.

To evaluate the security of the proposal it is also interesting to consider threats identified by NIST against ZTAs (Rose et al., 2020).

Subversion of ZTA decision process. The Subversion of ZTA Decision Process maps directly to attack A5 that alters the integrity of the decision process of the Access Control Engine. This attack is mitigated through BFT replication of the Access Control Engine.

Denial-of-Service or network disruption. The Denial-of-Service attack considered by NIST involves impeding availability of the Access Proxy and the Access Control Engine. The proposed solution mitigates unavailability of the Access Control Engine through BFT replication. Moreover, the Access Proxy can be geographically replicated with well-known techniques, thus mitigating unavailability.

Stolen credentials or insider threat. Insider threats or attackers that leverage stolen credentials are already considered and their impact mitigated. As discussed in Section 4, credential

theft can be mitigated through multi-factor authentication. Moreover, stolen credentials are not sufficient because the adversary requires control of the user device. Finally, we consider insider threats and mitigate possible attacks that they can launch, such as A7 and A3 + A6.

Visibility on the network. NIST identifies encrypted traffic as a limitation in network visibility through traffic inspection. The proposal preserves network visibility by allowing inspection of user requests received by the Access Proxy which terminates confidential communication channels.

Storage of system and network information. Another identified threat involves gaining reconnaissance information by violating the confidentiality of Sources (e.g. Access Policies database) and supporting systems (e.g. SIEM). Access to security critical components must be restricted to administrators only. Moreover, the proposed architecture is secure even against adversaries that have complete knowledge of the architecture by minimizing single points of vulnerability through survivable design practices.

Reliance on proprietary data formats or solutions. Lack of interoperability constitutes a relevant practical concern, especially in the context of survivable zero trust which relies on design diversity to mitigate common mode failures. Indeed, future challenges in the field of zero trust involve standardization efforts to guarantee interoperability and prevent vendor lock-in.

Use of non-person entities (NPE) in ZTA administration. Finally, the compromise of automated administration tools may allow attackers to perform unauthorized actions on the ZTA. As automated administration tools perform actions on behalf of administrators, they inherit their trust assumptions and thus it is essential to restrict access and protect them from abuse.

8. Feasibility of the architecture

In this section we analyze the performance of the proposal and discuss some limitations that could be addressed by future research.

8.1. Performance analysis

The proposal of an innovative architecture opens the important question of whether the overheads related to the survivable ZTA can be really applied to protect accesses to cloud computing environments.

We evaluate the overall response times of the two most expensive operations of the survivable ZTA from the user point of view (Section 3): *user authentication* and *request authorization*. We can anticipate that the proposed architecture is characterized by response times of less than two seconds for *user authentication* and in the order of some hundreds of milliseconds for *request authorization*. Hence, we can consider this architecture feasible from the performance point of view.

User authentication is based on the survivable SSO component. In Section 6.2, we identify PROTECT (Zhang et al., 2020) as the best candidate for our survivable ZTA. Considering realistic scenarios where the threshold of tolerable malicious servers is set to one or two units, PROTECT offers authentication response times in the order of two seconds even for low-powered devices. This timing is considered acceptable even with non-survivable access control architectures, where the security of password-based authentication is based on the adoption of password-hashing operations that introduce significant delays (Biryukov et al., 2021; Jarecki et al., 2018).

Request authorization is based on the survivable Access Control Engine and BFT databases. We first consider the performance of the survivable Access Control Engine, which relies on BFT state machine replication protocols, such as Babay et al. (2018); Garcia et al. (2016); Kirsch et al. (2013); Luiz et al. (2014). These proposals analyze the protocols by adopting a security threshold of $f = 1$ that complies with our recommendations for the Access Control Engine in Section 7. For example, the authors in Kirsch et al. (2013) implement an intrusion tolerant system by relying on the Prime protocol (Amir et al., 2010), which allows BFT state machine replication with strong upper bounds on response times. The authors evaluate performance in a local area network and show that requests can be processed with response times in the order of 35 ms. The same Prime protocol is adopted by the intrusion tolerant system proposed in Babay et al. (2018), where performance evaluations show that the system processes 99.999% of requests within a response time in the order of 100 ms in a wide area network. In Garcia et al. (2016) the authors adopt the MOD-SMaRt state machine replication protocol implemented through the BFT-SMaRt library (Bessani et al., 2014) that can process requests with an average response time of less than 10ms in a local area network. The Prime protocol is the best candidate to implement the Access Control Engine as a replicated state machine because it guarantees upper bounds on response times even in wide area networks, which is relevant in cloud applications, while the MOD-SMaRt protocol does not guarantee response times upper bounds. To analyze the performance of BFT databases, we consider MITRA (Luiz et al., 2014) that is based on the already mentioned BFT-SMaRt library) and HRDB (Vandiver et al., 2007). In Section 6.1 we identified them as the best trade-off solutions among the examined BFT databases. These proposals evaluate the performance with security threshold set to $f = 1$, which also com-

plies with our survivability recommendations for ZTA. Their response times are in the order of 200 ms (Vandiver et al., 2007).

As expected, survivable SSO appears as the component responsible for the highest response times of the ZTA. When a similar authentication is required, the total response time of a user request is typically below 2 s because modern devices have characteristics much better than those used in PROTECT experiments. It is worth to note that an SSO authentication must be executed only when a user initiates a session. Further requests can leverage the valid token which can be verified with negligible overhead with respect to Access Control Engine and BFT databases response times (Zhang et al., 2020). After the initial SSO authentication, the response time of each user request remains in the order of hundreds of milliseconds or below.

8.2. Possible limitations

We now summarize the design trade-offs of the proposed architecture and highlight limitations that may be addressed by future work. Our architecture relies on survivable databases to securely store data even in presence of intrusions. Proposals such as MITRA (Luiz et al., 2014) and HRDB (Vandiver et al., 2007) guarantee the best trade-offs in terms of security and performance among the examined databases (Section 6.1) and they are good candidates to obtain a real implementation. Each database has different assumptions about survivability: MITRA assumes a bounded amount of faulty replicas over the whole architecture lifetime; HRDB assumes a trusted component. These assumptions are acceptable trade-offs to guarantee survivability. Relaxing them deserves further research efforts in the field of survivable databases that go beyond the scope of this paper.

A relevant design aspect is represented by the reduction of the amount of trust in the Access Proxy by separating policy enforcement in a distinct component that we identify as the Resource Proxy. This choice has several security benefits (Section 5) but requires each resource to trust the corresponding Resource Proxy for correct policy enforcement. Ideally, the Resource Proxy should be designed as a survivable component so that resources can rely on untrusted components for policy enforcement. While we are not aware of any theoretical limit to this extension, trusting enforcement points seems an intrinsic requirement of access control architectures.

It is worth to note that the proposed architecture adopts a novel protocol guaranteeing end-to-end confidentiality of the response sent by the resource to the user. The proposed version of the Access Control Engine requires plaintext requests to evaluate access control policies and thus the protocol does not guarantee end-to-end request confidentiality. Adopting cryptographic protocols that allow to evaluate policies on encrypted requests would add significant performance overhead and would prevent full operation of intrusion detection systems. The design of a feasible protocol for end-to-end confidentiality with limited negative effects on performance and operation is an open research challenge that can be considered for future work even for different application contexts.

9. Conclusions

Zero trust architectures are emerging as solutions that solve the limitations of cyber defenses based on traditional perimeter for the protection of modern organizations. Existing designs assume that cyber attackers are confined to the data plane enforcing access control policies and that they cannot compromise the control plane components establishing access control policies. We present a novel survivable zero trust architecture that overcomes this strong assumption. The proposed design distributes trust among multiple control plane components so that the architecture can tolerate intrusions and recover from successful attacks. The proposed architecture is appropriate whenever combining cyber resilience and security is an important requirement, as in the case of critical infrastructures. This proposal is feasible even from the performance point of view, but it is open to future research improvements in the field of databases satisfying survivability, and survivable protocols that can combine end-to-end confidentiality and access control policy evaluation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit authorship contribution statement

Luca Ferretti: Conceptualization, Methodology. **Federico Magnanini:** Conceptualization, Investigation, Methodology, Writing – original draft. **Mauro Andreolini:** Methodology, Writing – review & editing. **Michele Colajanni:** Methodology, Supervision, Writing – review & editing.

REFERENCES

- Agrawal S, Miao P, Mohassel P, Mukherjee P. PASTA: password-based threshold authentication. *Proc. ACM Conf. Computer and Communications Security (ACM SIGSAC)*, 2018.
- Amir Y, Coan B, Kirsch J, Lane J. Prime: byzantine replication under attack. *IEEE Trans. Dependable Secure Comput.* 2010;8:564–77.
- Apruzzese G, Pierazzi F, Colajanni M, Marchetti M. Detection and threat prioritization of pivoting attacks in large networks. *IEEE Trans. Emerg. Top. Comput.* 2017;8:404–15.
- Ardagna CA, De Capitani di Vimercati S, Paraboschi S, Pedrini E, Samarati P. An XACML-based privacy-centered access control system. *Proc. First ACM Workshop on Information Security Governance*, 2009.
- Avizienis A. The methodology of *n*-version programming. *Softw. Fault Toler.* 1995;3:23–46.
- Avizienis A, Laprie JC, Randell B, Landwehr C. Basic concepts and taxonomy of dependable and secure computing. *IEEE Trans. Dependable Secure Comput.* 2004;1:11–33.
- Babay A, Tantillo T, Aron T, Platania M, Amir Y. In: *Ann IEEE/IFIP Int'l Conf Dependable Systems and Networks. Network-attack-resilient intrusion-tolerant SCADA for the power grid*; 2018.
- Baum C, Frederiksen TK, Hesse J, Lehmann A, Yanai A. In: *IEEE European Symp. Security and Privacy (EuroS&P). PESTO: proactively secure distributed single sign-on, or how to trust a hacked server*; 2020.
- Berenson H, Bernstein P, Gray J, Melton J, O'Neil E, O'Neil P. In: *ACM SIGMOD Record. A critique of ANSI SQL isolation levels*; 1995.
- Bessani A., Sousa J.a., Alchieri E.E.P. State machine replication for the masses with BFT-SMART. *Ann IEEE/IFIP Int'l Conf Dependable Systems and Networks* 2014.
- Bernstein PA, Goodman N. Serializability theory for replicated databases. *J. Comput. Syst. Sci.* 1985;31:355–74.
- Biryukov A, Dinu D, Khovratovich D, Josefsson S. In: *Internet Engineering Task Force. The memory-hard Argon2 password hash and proof-of-work function. Internet-draft draft-irtf-cfrg-argon2-13*; 2021. *Work in Progress*
- Boldyreva A.. Threshold signatures, multisignatures and blind signatures based on the Gap-Diffie-Hellman-Group signature scheme. *Int'l Workshop on Public Key Cryptography* 2003.
- Campbell M. Beyond zero trust: trust is a vulnerability. *Computer* 2020;53:110–13.
- Castro M, Liskov B. Practical byzantine fault tolerance and proactive recovery. *ACM Trans. Comput. Syst. (TOCS)* 2002;20:398–461.
- Clark J, Oorschot PCV. In: *IEEE Symp. Security and Privacy. Sok: SSL and HTTPS: revisiting past challenges and evaluating certificate trust model enhancements*; 2013.
- DeCusatis C, Liengtiraphan P, Sager A, Pinelli M. In: *IEEE Int'l Conf Smart Cloud (SmartCloud). Implementing zero trust cloud networks with transport access control and first packet authentication. IEEE*; 2016.
- Dhar S, Bose I. Securing IoT devices using zero trust and blockchain. *J. Organ. Comput. Electron. Commer.* 2020;31:18–34.
- Eidle D, Ni SY, DeCusatis C, Sager A. Autonomic security for zero trust networks. *IEEE 8th Ann. Ubiquitous Computing, Electronics and Mobile Communication Conference (UEMCON)*, 2017.
- eXtensible Access Control Markup Language (XACML) Version. 2013. <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>.
- Garcia M, Neves N, Bessani A. SieveQ: a layered BFT protection system for critical services. *IEEE Trans. Dependable Secure Comput.* 2016;15:511–25.
- Garcia R, Rodrigues R, Prego N. Efficient middleware for byzantine fault tolerant database replication. *Conf. Computer Systems (EuroSys)*, 2011.
- Gashi I, Popov P, Stankovic V, Strigini L. In: *Architecting Dependable Systems II. On designing dependable services with diverse off-the-shelf SQL servers. Springer*; 2004.
- Gashi I, Popov P, Strigini L. Fault tolerance via diversity for off-the-shelf products: a study with SQL database servers. *IEEE Trans. Dependable Secure Comput.* 2007;4:280–94.
- Gilman E, Barth D. *Zero Trust Networks: Building Secure Systems in Untrusted Networks*. O'Reilly Media, Inc; 2017.
- Jarecki S, Krawczyk H, Xu J. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. *Annual Int'l Conf. Theory and Applications of Cryptographic Techniques. Springer*, 2018.
- Kindervag J, et al. Build Security into your Network's DNA: The Zero Trust Network Architecture. Forrester Research Inc; 2010.
- Kirsch J, Goose S, Amir Y, Wei D, Skare P. Survivable SCADA via intrusion-tolerant replication. *IEEE Trans. Smart Grid* 2013;5:60–70.
- Kocher P, Horn J, Fogh A, Genkin D, Gruss D, Haas W, Hamburg M, Lipp M, Mangard S, Prescher T, et al. In: *Symp. Security and Privacy. Spectre attacks: exploiting speculative execution. IEEE*; 2019.

- Lee B, Vanickis R, Rogelio F, Jacob P. Situational awareness based risk-adaptable access control in enterprise networks. *Proc. Second Int'l Conf. Internet of things, Data and Cloud Computing and Association for Computing Machinery*, 2017.
- Lipp M., Schwarz M., Gruss D., Prescher T., Haas W., Mangard S., Kocher P., Genkin D., Yarom Y., Hamburg M.. MeltdownXiv preprint arXiv:1801.01202018.
- Lin C, He D, Huang X, Choo KKR, Vasilakos AV. BSeIn: a blockchain-based secure mutual authentication with fine-grained access control system for industry 4.0. *J. Netw. Comput. Appl.* 2018;116:42–52.
- Luiz AF, Lung LC, Correia M. In: *Int'l Symp. Network Computing and Applications. Byzantine fault-tolerant transaction processing for replicated databases*. IEEE; 2011.
- Luiz AF, Lung LC, Correia M. In: *ACM SIGMOD Record. Mitra: byzantine fault-tolerant middleware for transaction processing on replicated databases*; 2014.
- Maesa DDF, Mori P, Ricci L. Blockchain based access control. *IFIP Int'l Conf. Distributed Applications and Interoperable Systems*. Springer, 2017.
- Maesa DDF, Mori P, Ricci L. Blockchain based access control services. *IEEE Int'l Conf. Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2018.
- Maesa DDF, Mori P, Ricci L. A blockchain based approach for the definition of auditable access control systems. *Comput. Secur.* 2019;84:93–119.
- Molina HG, Pittelli F, Davidson S. Applications of byzantine agreement in database systems. *ACM Trans. Database Syst. (TODS)* 1986;11:27–47.
- Pedone F, Schiper N. Byzantine fault-tolerant deferred update replication. *J. Braz. Comput. Soc.* 2012;18:3–18.
- Reiter MK, Franklin MK, Lacy JB, Wright RN. The ω key management service. *J. Comput. Secur.* 1996;4:267–87.
- Rose S, Borchert O, Mitchell S, Connolly S. In: *Technical Report NIST Special Publication (SP) 800-207. Zero Trust Architecture*. National Institute of Standards and Technology; 2020.
- Schneider FB. Implementing fault-tolerant services using the state machine approach: a tutorial. *ACM Comput. Surv. (CSUR)* 1990;22:299–319.
- Schoenmakers B, Veeningen M. Universally verifiable multiparty computation from threshold homomorphic cryptosystems. *Int'l Conf. Applied Cryptography and Network Security*, 2015.
- Uttecht KD. In: *Technical report. Zero Trust (ZT) Concepts for Federal Government Architectures*. Massachusetts Inst of Tech Lexington Lexington United States; 2020.
- Vandiver B, Balakrishnan H, Liskov B, Madden S. In: *ACM Symp. Operating Systems Principles (SIGOPS). Tolerating byzantine faults in transaction processing systems using commit barrier scheduling*; 2007.
- Vanickis R, Jacob P, Dehghanzadeh S, Lee B. Access control policy enforcement for zero-trust-networking. *29th Irish Signals and Systems Conference (ISSC)*. IEEE, 2018.
- Ward R., Beyer B.. BeyondCorp: a new approach to enterprise security. *login*: 2014.
- Wu T, Malkin M, Boneh D. In: *USENIX Security Symposium. Building intrusion tolerant applications*; 1999.
- Zaheer Z, Chang H, Mukherjee S, Van der Merwe J. In: *Proc ACM Symp. SDN Research. eZTrust: network-independent zero-trust perimeterization for microservices*; 2019.
- Zhang Y, Xu C, Li H, Yang K, Cheng N, Shen XS. PROTECT: efficient password-based threshold single-sign-on authentication for mobile users against perpetual leakage. *IEEE Trans. Mob. Comput.* 2020;20:2297–312.
- Zhou L, Schneider FB, Renesse RV. COCA: a secure distributed on-line certification authority. *ACM Trans. Comput. Syst. (TOCS)* 2002;20:329–68.
- Zhu Y, Qin Y, Zhou Z, Song X, Liu G, Chu WCC. Digital asset management with distributed permission over blockchain and attribute-based access control. *IEEE Int'l Conf. Services Computing (SCC)*, 2018.

Luca Ferretti is assistant professor at the Department of Physics, Informatics and Mathematics of the University of Modena and Reggio Emilia (Italy). He received the Ph.D. degree at the International Doctorate School in information and communication technologies (ICT) of the same university. His research interests include information security and applied cryptography.

Federico Magnanini received the master's degree in computer engineering from the University of Modena and Reggio Emilia, Italy in 2019. He is working toward the Ph.D. degree at the International Doctorate School in information and communication technologies (ICT) of the University of Modena and Reggio Emilia, Italy. His research interests include information security and distributed ledger technologies.

Mauro Andreolini is an assistant professor at the Department of Physics, Informatics and Mathematics of the University of Modena and Reggio Emilia (Italy). He received his Master Degree (summa cum laude) at the University of Roma, Tor Vergata in 2001 and his Ph.D. in 2005 from the same institution. His research focuses on design, evaluation and security of distributed and cloud-based systems, based on a best-effort service or on guaranteed levels of performance.

Michele Colajanni is full professor in computer engineering at the University of Bologna. He received the Master degree in computer science from the University of Pisa, and the Ph.D. degree in computer engineering from the University of Roma in 1992. His research interests include security of large scale systems, performance and prediction models, Web and cloud systems.